

Student Name	Student Id
Volam Abhiraj	2024AA05622
Deepti Prashant	2024AB05171
Krishna kumar	2024AA05240
Ginny George	2024AA05275

GitHub Repository: <https://github.com/deepp11/heart-disease-mlops.git>

Project Overview

- Developed an end-to-end **heart disease classification** system using the UCI Heart Disease dataset with 14 clinical features and a binary target (disease vs no disease).
- Implemented a complete **MLOps pipeline** including data acquisition, preprocessing, model training with experiment tracking, CI/CD, containerization, Kubernetes deployment, and monitoring.

Setup and Installation

- Project is organized under content/mlops-heart-disease with subdirectories for src, api, docker, k8s, helm, monitoring, tests, and mlruns (MLflow artifacts).
- A **single requirements file** (requirements.txt) is generated for the Docker/API environment, including FastAPI, Uvicorn, scikit-learn, pandas, numpy, joblib, prometheus-client, and utilities such as httpx and python-multipart.

Typical setup steps:

📁 Project Structure

- `app/` - FastAPI application
- `deployment/` - Docker & Kubernetes configs
- `MLOps/` - Machine learning code
- `Dockerfile` - Container definition
- `requirements.txt` - Dependencies

⚡ Clone, setup, test, and deploy

- git clone <https://github.com/deepp11/heart-disease-mlops.git>
- cd heart-disease-mlops
- pip install -r requirements.txt
- ./run-api.sh & # Run in background
- sleep 5 # Wait for API to start
- ./quick_test.sh # Run tests
- ./simulate_load_balancer.sh # to simulate load balancer

🌐 API Endpoints

- Health: `curl http://localhost:8000/health`
- Docs: <http://localhost:8080/docs>

EDA and Modelling Choices

- Data loading and cleaning are encapsulated in `src/dataacquisition.py`, which downloads the UCI Heart Disease CSV, handles missing values, and ensures a clean DataFrame for the pipeline.
- A **preprocessing pipeline** (`HeartDiseasePreprocessor` in `src/preprocessing.py`) performs:
 - Separation of numeric and categorical features.
 - Imputation with `SimpleImputer` and scaling with `StandardScaler` for numeric columns, plus encoding and imputation for categoricals, all wrapped in a `ColumnTransformer` and `Pipeline`.
- The function `preparedatafortraining`:
 - Splits data into train/validation/test with stratification and separate test and validation ratios.
 - Fits the preprocessing pipeline on training data and transforms train/val/test into `X_train_processed`, `X_val_processed`, `X_test_processed`, returning them along with y arrays and the preprocessor object.
- Model development (`src/train.py`) includes:
 - Multiple classifiers: Logistic Regression, Random Forest, Gradient Boosting, and SVM with predefined hyperparameter grids (C, penalty, solver for Logistic Regression; `n_estimators`, `max_depth`, `min_samples_split` for Random Forest; `n_estimators`, `max_depth`, `learning_rate` for Gradient Boosting; and `kernel/C/gamma` for SVC).
 - `GridSearchCV` with ROC-AUC scoring, cross-validation, and selection of the best estimator per model type, followed by test-set evaluation with accuracy, precision, recall, f1-score, and ROC-AUC.

Include in your report:

- Rationale for using **ROC-AUC** as the primary metric (class-imbalance and ranking quality).
- Plots saved by training code: confusion matrix, ROC curve, and feature importance for tree-based models, stored as artifacts and logged to MLflow runs.

Experiment Tracking Summary

- **MLflow** is configured with a tracking URI `mlruns` and experiment name `heart-disease-classification` in `src/train.py`.
- For each model type:
 - A new MLflow run is started, tagged with `modeltype` and `cvfolds`.
 - Grid search best parameters are logged via `mlflow.log_params`, and cross-validation statistics (`cv_mean_roc_auc`, `cv_std_roc_auc`) are logged as metrics.
 - Test metrics (`test_accuracy`, `test_precision`, `test_recall`, `test_f1score`, `test_rocauc`) are logged using `mlflow.log_metric`.
 - Artifacts logged include confusion matrix image, ROC curve image, feature importance plot (for tree models), and a textual classification report.
 - The best model is serialized as an MLflow model artifact with `mlflow.sklearn.log_model` and also saved locally under `models/<model_name>/model.joblib`.
- A separate function `selectbestmodel` scans the results dictionary and selects the model with the greatest ROC-AUC, which is then combined with the preprocessor and saved as a deployment package (`models/bestmodel.joblib` or `modelpackage/models/fullmodelpackage.joblib`). In your report, summarize:
- Number of experiments (different model families, hyperparameter grid sizes).
- Which model was chosen as **best** by ROC-AUC and where its MLflow run can be found in `mlruns/`.

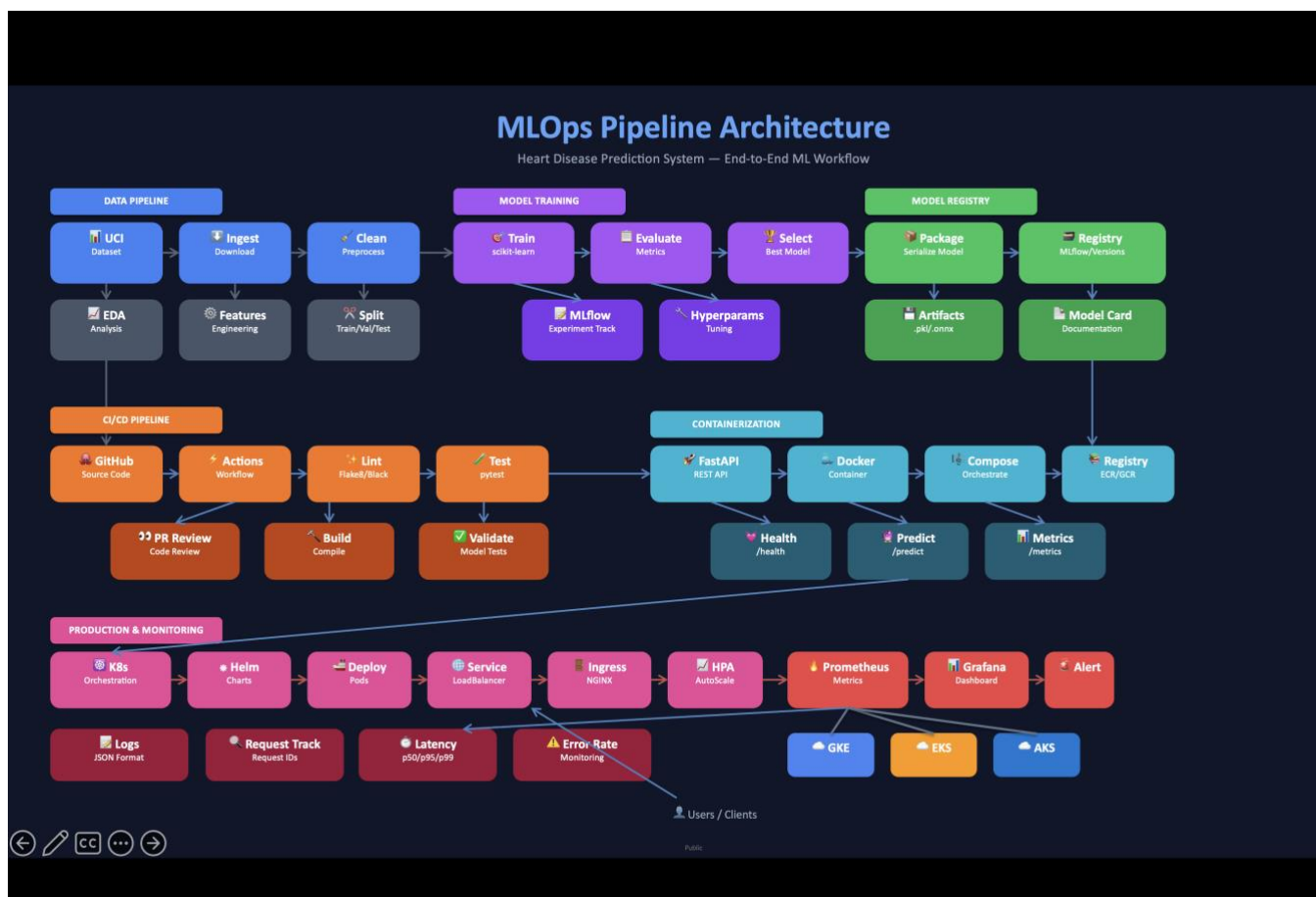
Architecture and System Design

- **High-level architecture (describe or draw diagram in report):**
 - Data acquisition/cleaning module → preprocessing module → training module with MLflow tracking → model package (model + preprocessor) → FastAPI serving container → Kubernetes deployment → monitoring stack with Prometheus and Grafana.
- **Training & packaging flow:**
 - `dataacquisition.loadandcleandata` → `preprocessing.preparedatafortraining` → `train.trainallmodels` → `train.selectbestmodel` → `train.savebestmodel` creates a portable deployment package used by the API.
- **Serving architecture:**
 - FastAPI app (`api/main.py`) loads the combined model-preprocessor package on startup from one of several candidate paths (`app/models/fullmodelpackage.joblib`, `content/mlops-heart-disease/modelpackage/models/fullmodelpackage.joblib`, etc.).

- API exposes endpoints: / (root info), /health, /modelinfo, /metrics, /predict, /predictbatch, with pydantic schemas PatientData, PredictionResponse, batch request/response models.
- **Infrastructure & deployment:**
 - Dockerfile under docker/Dockerfile (also copied to project root) uses python:3.10-slim, installs dependencies, copies api, src, models, and modelpackage, sets appuser, defines HEALTHCHECK for /health, and runs uvicorn api.main:app on port 8000.
 - docker-compose.yml defines api service and optional prometheus and grafana services under a custom mlops-network, mounting model volumes read-only and wiring health checks.
 - Kubernetes manifests under k8s (base + overlays) plus a Helm chart helm/heart-disease-api provide deployment, service, config, ingress, and environment configuration for running the API on Minikube or cloud clusters.

Your architecture diagram can show:

- Boxes for: Data acquisition, Preprocessing, Training + MLflow, Model Registry/Artifacts, API container, K8s/LoadBalancer/Ingress, Prometheus/Grafana; arrows showing data and metrics flow.



CI/CD, Testing, and Deployment Evidence

- **Testing and quality gates:**
 - Unit tests for data processing (tests/unittest_dataprocessing.py), model (tests/unittest_model.py), and API (tests/unittest_api.py).
 - Integration test for end-to-end pipeline (tests/integration/test_pipeline.py) and pytest configuration (pytest.ini, tests/conftest.py).
 - Linting and formatting with Flake8, Black, and isort configured via .flake8 and pyproject.toml.
- **CI/CD workflows (.github/workflows):**
 - ci-cd-pipeline.yml runs on push/PR:
 - Install dependencies, run linting (Flake8, Black, isort), run tests (pytest), run model training steps, build Docker image, and archive artifacts/logs.
 - test.yml focuses on unit/integration testing across the codebase.
 - model-validation.yml validates models (e.g., thresholds for metrics) before promoting them.
- **Containerization and local deployment scripts:**
 - build.sh builds the Docker image heart-disease-api:latest and prints run/test commands.
 - run.sh runs the container on port 8000, with health check instructions and log commands.
 - testapi.sh uses curl to hit /health, /modelinfo, and /predict with sample JSON.

Include screenshots of:

- GitHub Actions runs for CI/CD showing successful workflows, failed runs for demonstration of gating, and artifact/log views. MLOPS_Assign1-2.pdf+1

Monitoring, Logging, and Documentation Assets

- **Logging and monitoring modules:**
 - monitoring/logging/logger.py sets structured JSON logs with correlation IDs and request metadata.
 - monitoring/prometheus/metrics.py defines application-level metrics:
 - httprequeststotal, httprequestdurationseconds, modelpredictiontotal, modelpredictiondurationseconds, modelpredictionprobability, healthcheckstatus, apierrortotal.
- **Monitoring stack:**
 - Docker Compose monitoring file monitoring/docker-compose.yml launches Prometheus, Grafana, and Alertmanager, wired to scrape the API's /metrics endpoint.

- Prometheus config monitoring/prometheus/prometheus.yml and alert rules monitoring/prometheus/alertrules.yml set up scraping of the API job and alerts (APIDown, HighErrorRate, HighLatency, HighPredictionLatency, HealthCheckFailing).
- Grafana dashboard JSON monitoring/grafana/dashboards/heart-disease-api.json defines panels for total requests, predictions, health status, p95 latency, request/prediction rates, prediction distribution, latency percentiles, and error rates.
- **Monitoring documentation and samples:**
 - monitoring/README.md documents how to start the monitoring stack (./start-monitoring.sh), where to access Prometheus and Grafana, example PromQL queries, and notes on Kubernetes monitoring via monitoring/kubernetes/monitoring.yaml (ServiceMonitor & PrometheusRule).
 - monitoring/logging/samplelogs.txt provides example structured logs for inclusion in the report.

In the report, add:

- Screenshot of Grafana dashboard, Prometheus target status, and an example alert (even if simulated).
- Example JSON log line from samplelogs.txt illustrating correlation IDs and metrics.

Final Summary Report

=====

 FINAL SUMMARY REPORT

=====

DATASET OVERVIEW:

- Source: UCI Heart Disease Dataset (ID: 45)
- Original samples: 303
- Cleaned samples: 297
- Features: 20 (after preprocessing)
- Target: Binary (0=No Disease, 1=Disease)

PREPROCESSING STEPS COMPLETED:

1. Loaded data from UCI repository
2. Checked for missing values (-9 coded and NaN)
3. Created binary target variable
6. Encoded 8 categorical features
7. Scaled 5 numerical features
8. Split data: 237 train, 60 test samples

KEY FINDINGS FROM EDA:

- Class Distribution: 137 with disease, 160 without (46.1% prevalence)
- Most Important Features (based on correlation):
 1. oldpeak: +0.424 correlation
 2. age: +0.227 correlation
 3. trestbps: +0.153 correlation
- Data Quality: 297 clean samples ready for modeling
- Feature Matrix: 20 features for ML algorithms

DATA READY FOR MODELING:

- X_train shape: (237, 20)
- X_test shape: (60, 20)
- y_train shape: (237,)
- y_test shape: (60,)

 ANALYSIS COMPLETE - Dataset is ready for machine learning!

```
=====
=====
=====
1 2  MODEL 1: LOGISTIC REGRESSION
3 4
=====
=====
```

HYPERPARAMETER TUNING

Performing Grid Search with 5-fold CV...

 Tuning completed!

 Best Parameters:

C: 1

max_iter: 1000

penalty: l2

solver: saga

Best CV ROC-AUC: 0.8991

 TEST SET EVALUATION

Accuracy : 0.8000

Precision : 0.8333

Recall : 0.7143

F1-Score : 0.7692

ROC-AUC : 0.9297

 5-Fold Cross-Validation ROC-AUC: 0.8991 (± 0.0720)

=====

=====

 MODEL 2: RANDOM FOREST CLASSIFIER

=====

=====

 HYPERPARAMETER TUNING

Performing Grid Search with 5-fold CV...

 Tuning completed!

 Best Parameters:

max_depth: None

max_features: sqrt

min_samples_leaf: 2

min_samples_split: 5

n_estimators: 100

Best CV ROC-AUC: 0.8961

 TEST SET EVALUATION

Accuracy : 0.7833

Precision : 0.8000

Recall : 0.7143

F1-Score : 0.7547
ROC-AUC : 0.9152

 5-Fold Cross-Validation ROC-AUC: 0.8961 (± 0.0558)

=====

=====

 MODEL SELECTION & DOCUMENTATION

=====

=====

MODEL SELECTION RATIONALE:

1. Logistic Regression:

- Chosen as baseline model for interpretability
- Provides probability scores
- Linear decision boundary suitable for initial assessment

2. Random Forest:

- Chosen for comparison with non-linear capabilities
- Handles feature interactions automatically
- Robust to outliers and feature scaling

HYPERPARAMETER TUNING PROCESS:

- Used GridSearchCV with 5-fold stratified cross-validation
- Primary optimization metric: ROC-AUC
- Tuned key parameters for each model family
- Selected best parameters based on cross-validation performance

EVALUATION METRICS:

- Accuracy: Overall correctness
- Precision: Important for medical diagnosis (avoid false positives)
- Recall: Important for screening (avoid false negatives)
- F1-Score: Balanced measure of precision and recall
- ROC-AUC: Overall discriminative ability

FINAL MODEL RECOMMENDATION

Logistic Regression weighted score: 0.8038

Random Forest weighted score: 0.7895

 RECOMMENDED MODEL: Logistic Regression

Reasons:

1. Better weighted performance score
2. More interpretable coefficients
3. Simpler model with comparable performance

=====

=====

✅ MODEL DEVELOPMENT COMPLETED

=====

=====

📁 Models saved as 'final_logistic_regression.pkl' and 'final_random_forest.pkl'

=====

=====

🔬 MLFLOW EXPERIMENT TRACKING

=====

=====

📁 SETTING UP MLFLOW

✅ Experiment: Heart_Disease_Classification

📁 Tracking URI: file:./mlruns

📊 LOADING DATA

✅ Data loaded successfully!

Training samples: 237, Features: 20

Test samples: 60

=====

=====

🔧 EXPERIMENT 1: LOGISTIC REGRESSION

=====

=====

👤 Running: LR_Default

2026/01/05 15:52:19 WARNING mlflow.models.model: `artifact\_path` is deprecated. Please use `name` instead.

✅ Logged: Accuracy=0.800, ROC-AUC=0.929

📊 Run ID: c818d00197af493687d2669a03de3e09

👤 Running: LR_StrongReg

2026/01/05 15:52:24 WARNING mlflow.models.model: `artifact\_path` is deprecated. Please use `name` instead.

✓ Logged: Accuracy=0.817, ROC-AUC=0.908

📊 Run ID: 9ba9becedbf2417d8e987f5f597ef840

🏃 Running: LR_L1Reg

2026/01/05 15:52:26 WARNING mlflow.models.model: `artifact\_path` is deprecated. Please use `name` instead.

✓ Logged: Accuracy=0.800, ROC-AUC=0.919

📊 Run ID: df04448302a9485d86a63a84bdafef33b

🏃 Running: LR_WeakReg

2026/01/05 15:52:28 WARNING mlflow.models.model: `artifact\_path` is deprecated. Please use `name` instead.

✓ Logged: Accuracy=0.833, ROC-AUC=0.935

📊 Run ID: e50bcc158a6b48a2b5c2dfb7f50da179

=====

=====

🔬 EXPERIMENT 2: RANDOM FOREST

=====

=====

🏃 Running: RF_Default

2026/01/05 15:52:31 WARNING mlflow.models.model: `artifact\_path` is deprecated. Please use `name` instead.

✓ Logged: Accuracy=0.850, ROC-AUC=0.910

📊 Run ID: 3c1f31fba7154425b12c2eebdd3f69e4

🏃 Running: RF_Shallow

2026/01/05 15:52:34 WARNING mlflow.models.model: `artifact\_path` is deprecated. Please use `name` instead.

✓ Logged: Accuracy=0.833, ROC-AUC=0.917

📊 Run ID: 1fe71d6d04e949618c6a398d8f397f47

🏃 Running: RF_Deep

2026/01/05 15:52:37 WARNING mlflow.models.model: `artifact\_path` is deprecated. Please use `name` instead.

✓ Logged: Accuracy=0.833, ROC-AUC=0.907

📊 Run ID: a65d52225f604a6c96d40ad6b9baf08e

🏃 Running: RF_Conservative

2026/01/05 15:52:42 WARNING mlflow.models.model: `artifact\_path` is deprecated. Please use `name` instead.

✅ Logged: Accuracy=0.817, ROC-AUC=0.917
📊 Run ID: 3b1dbe93edf54c46a8c447b7c199958e

=====

=====

🔧 EXPERIMENT 3: GRID SEARCH TUNING

=====

=====

👤 Running: GridSearch_RandomForest
2026/01/05 15:52:55 WARNING mlflow.models.model: `artifact\_path` is deprecated. Please use `name` instead.
✅ Best params: {'max_depth': None, 'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 100}
📈 Best CV score: 0.8961
🎯 Test ROC-AUC: 0.9152
📊 Run ID: e64a8e86f9db4126ae7d7fde8d20f27c

=====

=====

📊 EXPERIMENT SUMMARY

=====

=====

📈 TOP 5 RUNS BY ROC-AUC:

Run Name	Model Type	ROC-AUC	Accuracy	F1-Score	Run ID
LR_WeakReg e50bcc158a6b48a	linear_models	0.9353	0.8333	0.8148	
LR_Default c818d00197af493	linear_models	0.9286	0.8000	0.7692	
LR_L1Reg df04448302a9485	linear_models	0.9185	0.8000	0.7600	
RF_Conservative 3b1dbe93edf54c4	ensemble_models	0.9174	0.8167	0.7925	
RF_Shallow 1fe71d6d04e9496	ensemble_models	0.9174	0.8333	0.8148	

Repository Link and Final Report Structure

The assignment asks for:

- GitHub repository link containing:
 - src/, api/, docker/, k8s/, helm/, monitoring/, tests/, mlruns/, Dockerfiles,